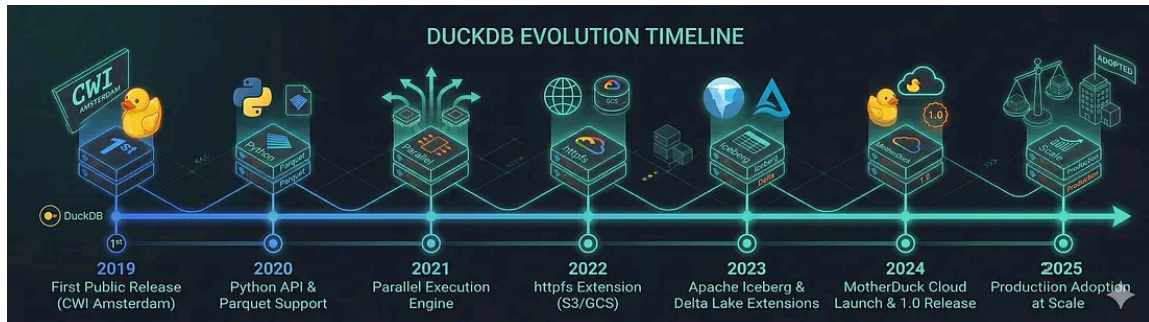


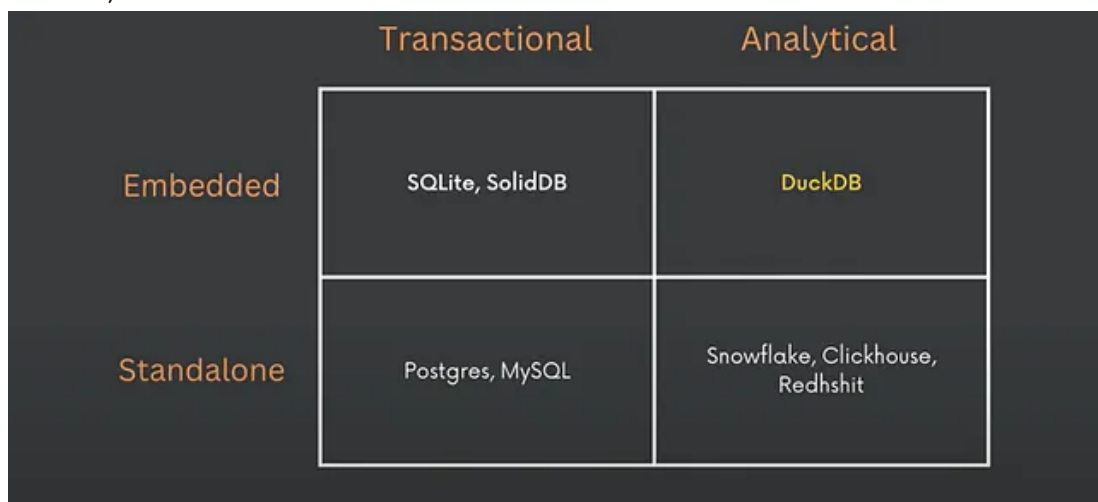
duckdb

DuckDB Lakehouse



DuckDB es presentado a menudo como el *SQLite* del análisis de datos:

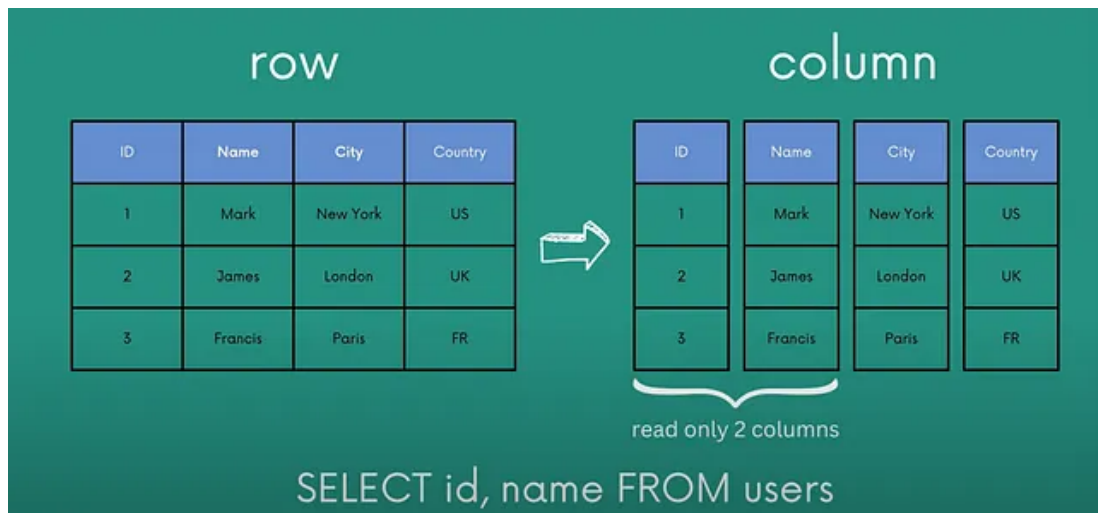
- Ejecuta en modo **in-process**, es decir, no es un servidor *stand-alone* al estilo de *MySQL*, *MariaDB*, etc.



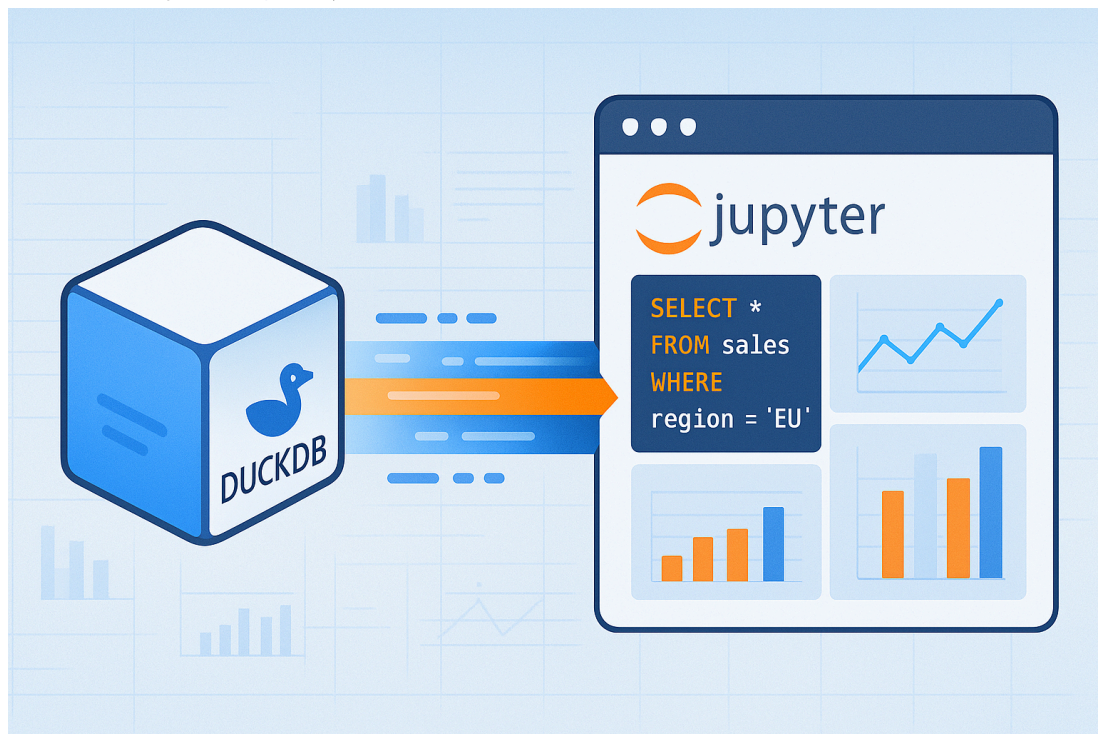
- Es muy ligero y fácil de instalar. Por ejemplo, en entorno *python* mediante un sencillo `pip install duckdb`
- A diferencia de *MySQL*, *MariaDB* o *SQLite* está optimizado para consultas de **OLAP** (agregados, joins, window functions) sobre datos en formato columnar
- Consecuentemente, su utilización **no es adecuada** para **OLTP**
- Está diseñado específicamente para el manejo de grandes volúmenes de datos. Ello lo convierte en una potente herramienta de **Big Data**
- Por su simplicidad y disponibilidad es perfecto para montar un **Data Lakehouse** local
- Es la navaja  suiza de la integración de datos (ver [lab S3 MinIO](#)) provenientes de todo tipo de **fuentes**

La magia de *DuckDB* proviene, sobre todo, de tres factores:

- Almacenamiento orientado a columnas frente al almacenamiento tradicional [orientado a filas](#)



- Ejecución vectorizada modo [OLAP](#)
- Integración con *Jupyter*, *Pandas*, *Polars*, *Arrow* y *Parquet* (Más info: [DataFrames](#), [DuckPandas](#) y [Scaling Tips](#))



Ejemplos Python API

Conexiones en memoria y persistente (base de datos en disco)



```
import duckdb

# In-memory database
con = duckdb.connect()

# Persistent database
con = duckdb.connect('mydatabase.db')
```

Ejecución de consultas básicas

```
# Creating a table
con.execute("CREATE TABLE sales (order_id INTEGER, product STRING, quantity
INTEGER, price_each DOUBLE, order_date TIMESTAMP, purchase_address STRING)")

# Inserting data
con.execute("INSERT INTO sales VALUES (1, 'Product A', 2, 19.99, '2023-01-01
10:00:00', '123 Main St')")
```

Integración con *Pandas* dataframes

```
import pandas as pd

# Querying data into a Pandas dataframe
df = con.execute("SELECT * FROM sales").fetchdf()

# Performing aggregations
revenue_per_product = con.execute("SELECT product, SUM(price_each * quantity)
AS total_revenue FROM sales GROUP BY product").fetchdf()
```



DuckDB vs Pandas

[EDA NYC Taxi dataset](#)

Integración de datos en formato *CSV/Parquet*



Game changer ⚡⚡

Importante: sin `LOAD DATA`. *DuckDB* consulta [CSV/Parquet](#) directamente
Siempre que sea posible, mejor utilizar *Parquet* que *CSV*

Datos en formato *CSV*

```
# Doc oficial: https://duckdb.org/docs/stable/data/csv/overview
#
```

```
import duckdb

# Consulta directa contra archivos CSV
result = duckdb.query("""
    SELECT category, AVG(price)
    FROM 'sales_2025.csv'
    GROUP BY category
    ORDER BY AVG(price) DESC
    LIMIT 10
""").to_df()
print(result)
```

Datos en formato *Parquet*

```
# Consulta directa archivo Parquet
# Doc oficial: https://duckdb.org/docs/stable/data/parquet/overview
#
parquet_file_path = 'path/to/your/file.parquet'
select_parquet_query = f"SELECT * FROM '{parquet_file_path}'"
parquet_data = con.execute(select_parquet_query).fetchdf()
print("Parquet File Data:")
print(parquet_data)

# Escritura a archivo Parquet
con.execute("COPY sales TO 'sales.parquet' (FORMAT PARQUET)")
```

Vistas contra datos en formato *CSV/Parquet*

```
# Crear la vista
#
file_path = 'path/to/your/file.parquet'

query_parquet = f"""
    CREATE OR REPLACE VIEW parquet_data AS
    SELECT * FROM read_parquet('{file_path}');
"""

con.execute(query_parquet)

# Consultar la vista
#
parquet_data = con.execute("SELECT * FROM parquet_data").fetchdf()
print("Parquet Data:")
print(parquet_data)
```

Lectura recomendada



La Rueda del Dharma (*Dharmachakra*)

TLDR: Los continuos vaivenes de la tecnología nos exigen un enfoque gobernado por la sensatez



[Building a data pipeline with DuckDB](#)

Labs



Lab 1: DuckDB en Docker

[Despliegue DuckDB WebUI en Docker](#)



Lab 2: DuckDB S3 MinIO

[Integración DuckDB S3 MinIO desde python](#)



Serie Python Labs

[Pandas, Polars, Arrow, DuckDB](#)

1. [Dataframes](#)
2. [Lazy Eval](#)
3. [Arrow + DuckDB](#)



Enlaces

[DuckDB In Action \(pdf\)](#)

[awesome-duckdb](#)

[DuckDB con DBeaver SQL IDE](#)